Observed Recoverable Behavioral Failure Modes in LLM Workflows

A Multi-Session Cross-Platform Case Study, Retry-Probe Pilot, and Multi-Model Evaluation Protocol

Vijay Suresh

Version 1.8 May 2026

Abstract

Standard large-language-model evaluations primarily score terminal outputs: whether a final answer is correct, safe, preferred, or useful. This paper studies a different class of failures, which we call *recoverable behavioral failures* (RBFs). An RBF occurs when a model appears to possess the capability, context, and tool access needed to complete a task but does not exercise that capability on the first pass, and subsequently succeeds after a brief retry, verification, or correction prompt that introduces no new substantive information. RBFs are observable in long, practical AI workflows including research synthesis, source verification, document drafting, code generation, multimodal interpretation, and artifact production. They are not captured by single-turn benchmark accuracy or by aggregate human preference data, because they only become visible across turns, when a second prompt reveals that the underlying capability was already available.

The paper makes five contributions. First, it defines recoverable behavioral failure as a distinct evaluation construct, separating capability from first-pass execution behavior. Second, it develops a layered sixteen-mode taxonomy spanning retrieval, reasoning, multimodal interpretation, infrastructure, artifact production, and editorial boundary control. Third, it reports a retry-probe pilot on a fifteen-item heterogeneous retrieval task in which first-pass resolution was 12/15 and after-retry resolution was 15/15, yielding a 20 percentage-point recoverability gap. Wilson 95% confidence intervals are reported for the small-sample rates, and an exact small-sample Fisher test associates first-pass failure with one structural item class (two-sided $p = 0.0022$, exact). Fourth, it extends the taxonomy with abstracted cross-platform workflow observations covering artifact-readiness, source-verification, audience-calibration, and editorial-boundary modes. Fifth, it specifies a controlled multi-model evaluation protocol grounded in a published task pack, with task families, deterministic and human-judged checks, sample-size guidance, an annotation rubric, and a reproducibility checklist. The pilot is exploratory; the multi-model protocol is offered as a planned replication rather than a completed study. We additionally report a small intra-vendor pilot result on the ten-task experiment pack across three Anthropic models (Sonnet 4.6, Opus 4.7, and Haiku 4.5), with per-model and per-category pass rates and Wilson 95% confidence intervals; these results are exploratory and intra-vendor only (cross-vendor extension is deferred to future work because OpenAI and Google credentials were unavailable at pilot time).

Keywords: large language models; evaluation; behavioral failures; retry behavior; agent reliability; tool use; artifact generation; self-verification; workflow readiness; case-study methodology; reproducible benchmark.

Contents

1 Introduction

Public discussion of language-model failures has converged on two broad categories. A *capability failure* occurs when the model lacks the knowledge, reasoning skill, perceptual ability, or tool fluency required to solve the task. An *alignment or safety failure* occurs when the model produces output that is harmful, deceptive, or otherwise off-policy. Existing evaluation infrastructure is built around these two categories: benchmark accuracy (Liang et al., 2022; Srivastava et al., 2023; Hendrycks et al., 2021) indexes the first; preference-based reinforcement learning (Christiano et al., 2017; Bai et al., 2022) and red-teaming index the second.

Many failures observed in practical AI workflows fit neither category cleanly. A user issues a request; the model declares part of the task unresolved, incomplete, or impossible; a short follow-up prompt elicits a substantively correct response without introducing new documents, tools, or facts. The capability was demonstrably available on the first turn. What changed between attempts was a search strategy, a verification step, a recognition of structural input, or an audit of an artifact for placeholders, formatting, or boundary leakage. The first attempt did not fail because the model could not do it. It failed because the model did not.

We refer to these events as *recoverable behavioral failures*. They are *behavioral* not in any anthropomorphic sense but because the failure is located in execution policy: effort allocation, persistence, verification, escalation, search discipline, multimodal reading, artifact finalization, and boundary management between conversation and deliverable. They are *recoverable* because a local correction reveals the model was capable of producing the correct output without additional information.

The distinction matters for modern AI workflows. Frontier models are now used for research synthesis, software debugging, document drafting, slide and spreadsheet generation, literature triage, citation checking, and tool-assisted operations. These are not single-turn tasks with a single final answer. They are multi-step, supervised processes in which the practical cost of a model is not only the probability of a wrong terminal answer; it is the amount of human oversight needed to detect premature stopping, recover incomplete artifacts, correct unsupported claims, and prevent process commentary from contaminating the final deliverable.

This paper is intentionally conservative. It does not estimate population-level failure rates for any vendor, model family, or product. It develops the construct, codes a naturalistic multi-session corpus, measures one mode quantitatively in a small pilot, extends the taxonomy to artifact-production and editorial-boundary failures observed in cross-platform professional workflows, and specifies a multi-model evaluation protocol that a future study can run to replicate and extend the findings.

1.1 Contributions

1. **Construct.** A definition of recoverable behavioral failure that separates capability from first-pass execution behavior, with operational measurement instruments (first-pass success, retry success, recoverability gap).

2. **Taxonomy.** A layered sixteen-mode taxonomy organized by failure locus.

3. **Pilot.** A retry-probe pilot with first-pass rate $12/15 = 80\%$ (Wilson 95% CI: $[54.8\%, 93.0\%]$), retry rate $15/15 = 100\%$, recoverability gap $+20$ percentage points (discordant-proportion CI: $[7.0\%, 45.2\%]$). Exact Fisher test on item-structural class association: two-sided $p = 0.0022$.

4. **Cross-workflow extensions.** Four additional modes from professional artifact-production and public-communication workflows.

5. **Multi-model protocol.** A controlled benchmark specification grounded in a ten-task pilot pack covering five task categories.

6. **Intra-vendor pilot results.** A multi-model run of the experiment pack across three Anthropic models (Sonnet 4.6, Opus 4.7, Haiku 4.5) producing per-(model, category) pass rates and Wilson 95% confidence intervals. The result is reported as a pilot with strong rubric-vocabulary caveats, not as a population estimate or model ranking.

2 Related Work

2.1 LLM benchmarks and evaluation

HELM (Liang et al., 2022), BIG-Bench (Srivastava et al., 2023), and MMLU (Hendrycks et al., 2021) established broad coverage of LLM capabilities across reasoning, knowledge, and language tasks. Their primary measurement target is the correctness, calibration, or preference of a terminal output under a fixed prompt. These benchmarks are essential and continue to drive frontier progress. They do not, however, separate the gap between what a model produces on the first attempt and what it can produce after an explicit retry, verification, or alternative search strategy. The present paper complements terminal-output evaluation by treating first-pass and post-retry performance as separately measurable quantities and by introducing artifact-level readiness checks alongside text-level correctness.

2.2 RLHF and preference learning

Reinforcement learning from human feedback (Christiano et al., 2017; Bai et al., 2022) and subsequent work on direct preference optimization aggregate pairwise human preferences over candidate responses. Preference labels are informative about which of two final answers is preferred, but they do not by themselves encode the reason for the preference. If the dispreferred answer would have improved under a follow-up retry without new information, that recoverability is not captured in the preference signal. As a result, a model trained to optimize aggregate preference can remain susceptible to behavioral failures whose cost is paid in turns of supervision rather than in measured preference deltas.

2.3 Tool-use and agentic benchmarks

SWE-bench (Jimenez et al., 2024) and WebArena (Zhou et al., 2024) evaluate complex multi-step tasks in software-engineering and web-navigation environments respectively. These agentic benchmarks measure end-to-end task completion under tool access. They do not always separate the intermediate decisions that constitute the agent's policy: when to retry a thin search, when to escalate to a different query construction, when to verify a numerical claim, when to ask for structural confirmation of a multimodal input, and when to declare an artifact complete. Recoverable behavioral failures are visible precisely at those intermediate decisions, which is why this paper introduces both retry-probe metrics and artifact-readiness checks.

2.4 Truthfulness, sycophancy, and deception

TruthfulQA (Lin et al., 2022) targets one class of behavioral failure: the assertion of false-but-popular claims. Work on sycophancy (Sharma et al., 2023) documents the tendency to align responses with stated user preferences even when those preferences diverge from the correct answer. Park et al. (2024) survey AI deception and adjacent confidently-wrong behavior. The present taxonomy intersects this literature but extends to a complementary surface pattern: the model may also decline, truncate, misread, or produce an incomplete artifact rather than confidently assert a false proposition. These behaviors can be equally costly in real workflows even when they do not look like classical hallucination.

2.5 Time-sensitive QA and knowledge staleness

Time-sensitive question answering (Chen et al., 2021; Liska et al., 2022) documents that static model knowledge degrades as the world changes. The present paper extends this concern from factual decay to procedural decay. When a model confidently describes a third-party tool interface that has been restructured, the resulting failure is not a factual hallucination in the traditional sense; it is a stale procedural assumption that wastes user effort. The proposed mitigation, confidence-decay framing, is the procedural analogue of well-known calibration strategies on factual outputs.

2.6 Long-horizon agent reliability and human-AI workflow reliability

A growing body of empirical and observational work documents the gap between single-turn capability and multi-turn reliability. Agent benchmarks report success rates that decline monotonically with task length, number of tool calls, and number of subgoals, even when single-step accuracy on the same operations is high. Studies of human-AI collaboration on knowledge-intensive tasks document substantial supervisory load placed on users when assistants fail in recoverable but uncaught ways: users absorb the cost of detecting premature stopping, of re-issuing corrective prompts, and of auditing artifacts whose surface form looks acceptable. Work on writing-assistance, coding-assistance, and research-assistance settings repeatedly finds that the time savings attributable to AI assistance are partly recouped by the time spent verifying outputs the assistant produced with insufficient self-checking.

The construct of recoverable behavioral failure offers a unified measurement language for these observations. Each subgoal in a long workflow is itself a candidate first-pass decision: the agent may search, verify, escalate, finalize, or audit, and may do so well or poorly. The practical reliability of the workflow is the cumulative product of those decisions. Reporting first-pass and retry-after-prompt rates separately, and augmenting with artifact-readiness and boundary-contamination checks, converts the informal supervisory-load observation from human-AI collaboration into a set of measurable rates that can be tracked over time and compared across models.

2.7 Case-study methodology

The empirical structure of this paper follows the theory-building case-study logic in Yin (2018). Naturalistic cases are appropriate when the phenomenon is multi-turn, workflow-dependent, and not yet reducible to isolated benchmark questions. The role of the case study is not to replace

controlled experimentation. It is to discover and characterize constructs that can then be measured through controlled replication. The retry-probe pilot in section 7 and the multi-model protocol in section 10 implement that subsequent measurement step.

3 Conceptual Framework

We define a recoverable behavioral failure relative to a verifiable task. Let an item i in task set T have a verifiable target outcome y_i. Let $A_1(i)$ denote the model's first-pass attempt under the original prompt, and let $A_r(i)$ denote the model's attempt after a standardized retry prompt that supplies no new substantive information — no new documents, no new tools, no new facts, no new task specification, no relaxed evaluation standard. Let $S_1(i) = 1$ when $A_1(i)$ satisfies the verification criterion, and analogously $S_r(i)$ for the retry attempt.

3.1 Core quantities

$$\text{First-pass success rate:} \quad p_1 = \frac{1}{N}\sum_{i=1}^{N} S_1(i) \tag{1}$$

$$\text{Retry success rate:} \quad p_r = \frac{1}{N}\sum_{i=1}^{N} S_r(i) \tag{2}$$

$$\text{Recoverability gap:} \quad G = p_r - p_1 \tag{3}$$

$$\text{RBF indicator for item } i\text{:} \quad \mathbb{1}[S_1(i) = 0 \ \wedge \ S_r(i) = 1] \tag{4}$$

under the standardized retry. Aggregating the indicator across items yields the empirical RBF rate.

3.2 Construct boundaries

The construct is intentionally conservative. The following are *not* counted as RBFs:

- Retries in which the user supplies a missing fact, document, or answer (*ordinary interactive correction*).
- Retries in which the task specification or evaluation standard is relaxed (*specification slippage*).
- Retries in which new tool affordances become available between attempts (*tool unlock*).
- Cases where the first-pass output is correct under a reasonable reading and the user demands an unnecessary rephrase (*preference rework*).

These exclusions protect the construct from inflation.

3.3 Additional measurable constructs

- **Batch-scaling degradation.** The change in per-item first-pass success as batch size N varies.

- **Artifact-readiness failure.** A deliverable exists but does not meet user-facing readiness criteria.
- **Boundary-contamination failure.** Process commentary appears inside a deliverable that should be publication-ready.
- **Verification compliance.** Proportion of items for which a stated verification step is performed when implied by the task.

4 Data and Methodology

4.1 Evidence base

The evidence combines a fourteen-session naturalistic case study of multi-turn workflows (document-generation, source-verification, multi-step research, quantitative reasoning, structured-visual interpretation, publication-preparation) with a cross-platform abstraction over additional professional workflows. The paper reports only abstracted workflow categories and failure mechanisms, with no individual identifying details.

4.2 Evidence tiers

Observations are assigned to evidence tiers to prevent overstating individual claims:

- **Tier 1**: Measured pilot item outcomes (quantitative PTS result).
- **Tier 2**: Transcript-verifiable multi-turn observations (mode definitions, examples).
- **Tier 3**: Abstracted cross-platform workflow summaries (artifact and editorial-boundary extensions).
- **Tier 4**: Hypothesized mechanisms requiring controlled replication.

4.3 Coding procedure

Each candidate event is coded with: (a) task context; (b) original output; (c) user retry; (d) post-correction outcome; (e) whether new substantive information was introduced; (f) failure mode; (g) trigger; (h) recovery type; (i) user-visible cost. Conservative coding rule: if recovery requires new information from the user, the event is excluded from RBFs.

4.4 Model-attribution caveats

The Claude.ai conversation export schema does not record per-message model identity. Attribution in the case study is reconstructed from user recall and contextual cues. The paper therefore avoids strong model-comparison claims based on the case study; the multi-model protocol in section 10 is the appropriate vehicle for controlled model comparison.

4.5 Responsible data handling

Raw transcripts are not released. The accompanying benchmark package contains items synthesized from neutral domains and does not incorporate user-specific identifiers. Future public releases should follow a two-layer model: a public benchmark from neutral sources, and a private audit appendix for trusted reviewers.

5 Taxonomy of Recoverable Behavioral Failures

The taxonomy is layered by failure locus. Locus separation matters because not every user-visible failure should be attributed to the model core. Some failures arise in retrieval policy, some in batch-effort allocation, some in artifact generation, some in client transport, some in product memory, some in the evaluation pipeline itself. A useful framework should locate the failure precisely enough to guide the right mitigation.

5.1 A note on mode overlap and granularity

Sixteen modes is a deliberate choice. Several pairs of modes are clearly adjacent and could in principle be collapsed: PTS and SMI are both retrieval-policy failures; MRT, UCT, and MSI are interruption-style failures at different loci; AFF and EBC are both artifact-side failures. We keep them separate for three reasons. (i) The *mitigation* implied by each mode is different. (ii) The *locus* attribution is different across apparent pairs. (iii) An empirically-driven collapse should follow from data: once cross-model and cross-user data accumulate, modes that always co-occur in the same items can be merged, while modes that dissociate empirically should stay separate. The present taxonomy is offered as a starting partition for measurement.

5.2 Premature Termination of Search (PTS)

Definition. The model declares an information item unresolved after a small number of retrieval attempts, even though a readily available alternative retrieval strategy would resolve it. The first attempt typically uses a thin or generic query construction; the second attempt, issued under a retry prompt that introduces no new information, succeeds.

Observed pattern. In the pilot retrieval task, three items whose URLs contained opaque numeric identifiers were marked unresolved on the first pass after generic keyword queries returned thin results. After a standardized retry prompt the model used site-targeted query construction and resolved all three. The capability gap on the first pass was therefore a query-construction policy decision rather than a knowledge or tool limit.

Why it matters. PTS is among the most consequential modes because it masquerades as missing knowledge. The user receives an 'unknown' response and reasonably treats it as a capability boundary. In fact the system has not exhausted available retrieval policy. The cost is paid both as missed information and as user-side supervisory load: the user must recognize that the response is recoverable and issue the retry themselves.

Table 1: Layered taxonomy of recoverable behavioral failure modes.

Code	Name	Primary locus	User-visible cost
PTS	Premature Termination of Search	Model / retrieval policy	False unresolved answers
BDTF	Breadth-Depth Tradeoff Failure	Model policy under batch load	Shallow per-item analysis
SMI	Search Methodology Inconsistency	Retrieval strategy	Inconsistent escalation
MRT	Mid-Response Truncation	Infrastructure	Lost progress
MSI	Mobile Streaming Interruption	Client / transport	Dropped responses
FPG	Feedback Persistence Gap	Product / training pipeline	Corrections not preserved
RFI	Retry-Failure Invisibility	Evaluation pipeline	Cannot-solve vs. did-not-try
STK	Stale Third-Party Tool/UI Knowledge	Model knowledge / tool guidance	Confident wrong UI paths
QI	Quantitative Inconsistency	Reasoning / self-audit	Contradictory numbers
PPG	Phantom Placeholder Persistence	Artifact / code generation	Placeholders as commands
UCT	Usage-Cap Termination	System / rate limit	Degraded resume state
VMC	Visual Misreading Cascade	Multimodal interpretation	Cascading misreadings
AFF	Artifact Finalization Failure	Artifact generation	File exists but unusable
SVD	Source Verification Drift	Citation / retrieval practice	Weakly supported claims
EBC	Editorial Boundary Contamination	Artifact boundary management	Process notes leak in
ACM	Audience / Context Miscalibration	Communication / register	Off-register content

Differentiation. PTS is distinct from **SMI** (which concerns inconsistency across similar subtasks rather than premature stopping on one subtask) and from **RFI** (which is the evaluation-pipeline blind spot that makes PTS invisible in offline review). PTS is also distinct from genuine retrieval failure where no alternative strategy would help; the diagnostic for PTS is that an alternative strategy demonstrably exists.

Detection. Retry probes constructed so that the first-pass query is likely to be thin but at least one alternative strategy is known to succeed. Measure first-pass success and retry-after-prompt success separately; the gap quantifies PTS. Probe construction recipes include items with opaque-identifier URLs, items whose canonical answer is on a specific known site, and items that require an exact-string match the generic query would dilute.

Mitigation. An explicit retrieval escalation policy: when the first search returns thin, the model should systematically try query rewriting, site restriction, exact-string identifier search, and source triangulation before declaring failure. The escalation should be visible to the user so that supervision is calibrated rather than always assumed; this also helps the user trust the 'unknown' verdicts that remain after escalation.

5.3 Breadth-Depth Tradeoff Failure (BDTF)

Definition. When presented with a large batch, the model implicitly optimizes for covering all items rather than verifying each item deeply. The result is a superficially complete response with weak per-item grounding: every item appears addressed, but per-item depth degrades as the batch size grows.

Observed pattern. Multi-item evaluation tasks elicit short, uniform dispositions even when individual items differ materially in difficulty. Items requiring secondary lookup are sometimes resolved without the lookup having occurred. Items requiring verification are sometimes presented with confident framings derived from partial reads. The user infers uniform depth from uniform output length, which is exactly the inference the model has not earned.

Why it matters. BDTF is an allocation error in which the appearance of coverage substitutes for verification. The user reads a list, sees that every item has been addressed, and reasonably assumes uniform depth. Without explicit depth indicators the user is forced to re-verify each item to recover trust—at which point the breadth advantage of the batch was illusory and the depth cost has merely shifted.

Differentiation. BDTF differs from **MRT** (which is truncation within a single response) and from **PTS** (which is premature stopping on a single retrieval). BDTF is a batch-level allocation phenomenon: per-item correctness degrades as N grows even when per-item tasks at small N are reliably solved. It can co-occur with PTS but is operationally separable through batch-size variation.

Detection. Batch-scaling curves: administer the same items at batch sizes N = 1, 5, 15, 30. Measure per-item correctness against N. The slope quantifies BDTF; the inflection point identifies the effective batch capacity. A healthy policy under load surfaces verification depth per item rather than hiding allocation behind uniform output length.

Mitigation. Batch-aware effort budgeting: for large batches, the model should state per-item uncertainty, allocate verification depth, and surface items that received only shallow processing. A simple product affordance is per-item flags that distinguish 'verified,' 'partially verified,' and 'not verified' outputs.

5.4 Search Methodology Inconsistency (SMI)

Definition. The model uses stronger retrieval tactics for some items and weaker tactics for similar items within the same session, with no explicit reason for the difference. Query construction varies between generic keyword search, site-targeted search, and identifier-anchored search without a stated escalation rule.

Observed pattern. Across a batch of retrieval subtasks, items that would have benefited from the stronger tactic receive the weaker tactic; users notice the inconsistency only when they observe

that one item succeeded under exactly the strategy that was not applied to a similar item. The same model can exhibit PTS on one item, escalate appropriately on a similar item, and fail to apply the same escalation to a third.

Why it matters. Inconsistent retrieval discipline shifts the burden of escalation onto the user. A robust workflow expects a predictable escalation protocol on thin first results: query rewriting, site restriction, identifier search, title extraction, source triangulation. SMI occurs when this escalation is selectively applied, so the user cannot rely on uniform retrieval policy and must monitor strategy choice item by item.

Differentiation. PTS is a failure mode within a subtask; SMI is a consistency mode across subtasks. The two can co-occur but are operationally separable: SMI persists even when overall PTS incidence is low, because the issue is the conditional probability of strategy escalation given a thin first result, not the absolute incidence of premature stopping.

Detection. Per-subtask query logging within a session. Compute (a) the distribution of queries-per-subtask, (b) the distribution of query-construction strategies, and (c) the conditional probability of switching strategy given a thin first result. A healthy retrieval policy should exhibit consistent escalation across structurally similar subtasks.

Mitigation. A documented escalation protocol applied uniformly across retrieval subtasks. Internal logging that exposes the chosen strategy to evaluators and, where appropriate, to the user, so that inconsistencies are noticeable rather than hidden. Consistency is itself a user-trust signal even when the underlying strategy is imperfect.

5.5 Mid-Response Truncation (MRT)

Definition. The model begins generating a response, makes partial progress, and the stream terminates before completion, often after partial tool use or partial drafting. The termination is signaled by a generic error rather than a graceful wind-down.

Observed pattern. Long multi-part responses on heterogeneous batches sometimes terminate with a generic recovery message. Partial work is not always preserved or replayable. The user is required to issue a fresh prompt to resume, with no native carry-over of intermediate state. In long workflows the cost of lost intermediate progress compounds: each truncation imposes recovery work proportional to task complexity.

Why it matters. MRT is costly not only because the answer is incomplete but because the partial work may need to be reassembled manually. The user must reconstruct what the model had already done, decide what to keep, and re-prompt for the remainder. The cost scales with response length and tool-call density—exactly the regime where users were relying on the model to reduce supervisory load.

Differentiation. MRT is an infrastructure-side termination of a single response; **UCT** is a system-level rate-limit termination of the conversation; **MSI** is a client-side stream drop. All three share the user-visible appearance of an interrupted task but differ in locus, signal, and recovery behavior.

Detection. Near-budget telemetry: log the fraction of token, tool-call, or wall-clock budget consumed at response termination. For a healthy generator, the distribution should be bimodal—very early when the task is short, well below threshold when the model voluntarily concluded. A spike of terminations at or near the budget boundary indicates MRT.

Mitigation. A structured progress ledger that allows the model and the system to preserve and resume intermediate state when generation fails. Resume-state checkpoints during long generations, with the option to surface and replay partial outputs. Where graceful termination is feasible, the model should be cued to summarize completed work before the budget is exhausted.

5.6 Mobile Streaming Interruption (MSI)

Definition. Mobile or client-side interruptions can drop an in-progress streaming response, with no recovery of the partial output when the user returns to the session. Screen-lock, backgrounding, and connection drops on mobile clients are common triggers.

Observed pattern. Screen-lock during streaming generation on mobile clients sometimes terminates the connection. The session reopens without the partial response surfaced. Users describe the failure as workflow unreliability rather than as a model error—they may not know whether the model finished and the partial output was lost in transit, or whether the model itself never finished.

Why it matters. Long AI workflows increasingly depend on streaming responses. When the client loses partial output, the cost is workflow-level: the user must re-issue the request and absorb the full latency again. The failure compounds with MRT and UCT to make long sessions on mobile materially less reliable than on desktop.

Differentiation. MSI is client/transport-layer; MRT is infrastructure/model-side; UCT is system rate-limit. All three are recoverable in principle through different mechanisms—client-side buffering for MSI, server-side resume for MRT, graceful continuation for UCT—but the locus determines the correct mitigation.

Detection. Client-side telemetry on stream-drop events correlated with foreground/background lifecycle and with screen-lock events. The relevant rate is interrupted streams per long-generation request on mobile clients, segmented by client version and connection type.

Mitigation. Persistent server-side response buffering with client reattachment on resume, so a screen-lock event does not erase work already produced. Notification surfaces that allow the user to

recover the interrupted response on next session open, with the partial output highlighted so the user can decide whether to continue or restart.

5.7 Feedback Persistence Gap (FPG)

Definition. Within-session corrections, scope clarifications, and explicit user preferences do not persist beyond the session unless an explicit memory surface or product mechanism captures them. The same user encounters the same correctable issue in subsequent sessions.

Observed pattern. A user corrects a stylistic or factual error in one session; the same error recurs in subsequent sessions without the correction being applied. Aggregate signal from preference data eventually reaches model training, but individual users observe no compounding return on their corrective effort across sessions.

Why it matters. FPG is not always a defect; user control and privacy require limits on cross-session persistence. The evaluation issue is that repeated corrections impose longitudinal cost and are invisible in single-session benchmarks. The same user who invests heavily in shaping behavior receives the least cumulative benefit absent an explicit memory mechanism—an incentive misalignment for power users.

Differentiation. FPG is the product/training-pipeline analogue of within-session retry behavior. It is distinct from FPG-adjacent caching mechanisms because it concerns user-corrective signal, not retrieval reuse. It is also distinct from explicit memory features whose presence renders FPG inapplicable for the affected facts.

Detection. Out of scope for single-session response-level evaluation. The appropriate measurement is longitudinal: time-to-recurrence of a corrected error, and the proportion of stable preferences that survive across sessions for a given user. These metrics require user-level instrumentation and informed consent.

Mitigation. Explicit per-user memory surfaces with user-visible controls. Treating correction events as weighted training signal scaled by user-investment proxies (turn count, correction explicitness). Longitudinal user-level evaluation metrics that surface the FPG cost so it can be tracked and reduced over time.

5.8 Retry-Failure Invisibility (RFI)

Definition. Standard offline evaluation review records a failed turn but not the counterfactual that the same model would have succeeded under a standardized retry prompt. As a result, capability failures and recoverable behavioral failures are statistically conflated in evaluation pipelines.

Observed pattern. An evaluator reviewing transcripts sees a model declare an item unresolved and codes it as a capability failure. The fact that a follow-up prompt with no new information produced a correct answer is not consistently captured. Without explicit retry capture, PTS, BDTF, SMI, and VMC all manifest in transcripts as terminal failures that look like capability gaps.

Why it matters. Without a measurement that isolates retry behavior, the behavioral subset is not separable from the (much larger and more studied) capability-failure population. This is the central evaluation-pipeline blind spot this paper documents. Recoverability remains invisible as long as offline review does not run the counterfactual retry.

Differentiation. RFI is the only mode in this taxonomy whose locus is the evaluation pipeline itself, not the model or the system. It is in the taxonomy because mitigating the other modes requires first making them measurable, and that measurability is what RFI denies.

Detection. Counterfactual replay: for any flagged failed turn in offline review, replay the same prompt with explicit retry framing. If success rate jumps materially under retry, the original failure was behavioral, not capability-bound. The behavioral subset can then be re-coded against the rest of the taxonomy.

Mitigation. Augment offline evaluation pipelines with retry traces. Distinguish first-pass and retry-after-prompt success in dashboards and headline metrics. Track recoverability gap as a first-class evaluation quantity reported alongside accuracy. This change is infrastructural rather than model-side and can be implemented today.

5.9 Stale Third-Party Tool/UI Knowledge (STK)

Definition. The model provides confident instructions for third-party tools or interfaces whose live UI has changed since training cutoff. Instructions are plausibly worded but cannot be followed because labels, menus, or setup flows differ from the model's stale knowledge.

Observed pattern. Multi-turn back-and-forth in which the user reports the live UI does not match the model's description. The model corrects only after the user describes the current UI in detail. The cost is paid as user-time spent navigating an interface the model has misdescribed, and as the user-side burden of recognizing that the model's confident instructions are stale.

Why it matters. Fast-evolving SaaS surfaces have UI-change cadences shorter than typical training-cutoff lags. The cost of stale UI knowledge is real and is paid disproportionately on developer-facing tools whose menus and labels evolve quickly. Unlike factual hallucination, the surface form of the answer is plausible; only by trying to follow the instructions does the user discover the staleness.

Differentiation. STK is procedural staleness; classic time-sensitive QA is factual staleness. STK is distinct from PTS (which is about effort allocation on a single retrieval) and from QI (which is

about internal numerical contradiction). The user-visible signal is the same as a hallucination, but the locus is staleness, not invention.

Detection. Confidence-decay scoring: any output containing third-party-tool navigation instructions should be wrapped with explicit staleness language proportional to the elapsed time since training cutoff weighted by the target service's historical UI-change cadence. A registry of high-churn services makes this practical.

Mitigation. Default verification framing on tool instructions: present the path as an estimate, suggest the user confirm against current documentation, and offer to update if the user describes the live UI. The framing converts STK from a confident failure into a collaborative verification step.

5.10 Quantitative Inconsistency (QI)

Definition. Within a single response, the model emits two or more numerical claims that contradict each other under the response's own stated assumptions. The contradiction is internally checkable but is not self-audited before delivery.

Observed pattern. An analytic response contains a breakeven figure and an average-cost figure that cannot both be true under the response's own stated unit cost. Users with quantitative fluency catch the inconsistency; users without may rely on the inconsistent analysis. The model owns the error when prompted, but the absence of any self-audit before returning is the behavioral failure.

Why it matters. QI is especially costly because the answer looks analytical and authoritative. Internal contradiction can propagate into downstream artifacts (slides, spreadsheets, business plans) where it is harder to detect after the fact. The user-visible signal is conscientiousness; the underlying behavior is the absence of arithmetic verification.

Differentiation. QI is reasoning/self-audit-side. It is distinct from PTS (retrieval-side) and SVD (source-grounding-side). QI does not require any external information to detect; it requires intra-response arithmetic verification under the response's own stated assumptions.

Detection. Intra-response consistency checks: programmatically extract numerical claims, their stated relationships, and verify arithmetic consistency under the response's own assumptions. The check is mechanical and cheap; it can run model-side as a self-audit or system-side as a post-pass.

Mitigation. An explicit numeric self-audit pass before returning responses with two or more quantitative claims. Where the audit fails, the model either revises or surfaces the inconsistency to the user. The audit is computationally trivial relative to the generation that produced the claims.

5.11 Phantom Placeholder Persistence (PPG)

Definition. The model issues a command, file path, citation field, configuration value, or code snippet containing an unresolved template placeholder even when the surrounding context implies a concrete value is needed. The placeholder substring is emitted as if it were a final value.

Observed pattern. A shell command emitted with a literal placeholder substring; a citation block with a template author or year field; a code snippet with a generic project path. The user pastes the command and receives an immediate error. The fix is mechanical—substitute the context-supplied value—but the responsibility was silently shifted to the user.

Why it matters. PPG silently shifts work to the user. Placeholders that should have been resolved from context are emitted as final output with no warning that substitution is required. The cost is small per instance but is paid every time the user runs the command and discovers the failure on execution.

Differentiation. PPG is artifact-side, not reasoning-side. It does not require the model to know any additional facts; it requires the model to recognize when emitted output is meant to be runnable versus illustrative, and to substitute or warn accordingly.

Detection. Regex-detect placeholder patterns in code-block output: angle-bracketed tokens, path-template strings such as 'path/to' or 'your-', and common template names. Each match should trigger either substitution from session context or an explicit user-substitution prompt.

Mitigation. A pre-delivery scan of code and command outputs for placeholder patterns. When found, substitute from session context if context unambiguously supplies the value; otherwise mark the placeholder explicitly and prompt the user. The check is deterministic and cheap.

5.12 Usage-Cap Termination (UCT)

Definition. A rate limit or usage cap terminates the conversation mid-task. The system surfaces a user-facing message but recovery often degrades to a malformed completion rather than graceful state preservation. The continuation turn following the cap event does not reliably resume the task.

Observed pattern. After a long agentic stretch, the model emits a usage-cap notice. Subsequent continuation prompts elicit a short, non-substantive completion that does not resume the task. The pattern can recur within a single session under sufficient agentic load, compounding workflow disruption.

Why it matters. UCT is system-level, but its user-visible effect is workflow-integrity loss. A failure to resume cleanly turns a rate-limit event into a workflow disruption disproportionate to the actual cap. The user loses not only the throttled tokens but also the continuity that depended on resuming where the session paused.

Differentiation. UCT is rate-limit-driven; MRT is budget-driven within a single response; MSI is client/transport-driven. The mitigation surfaces differ—graceful resume design for UCT, structured progress for MRT, client buffering for MSI—but the user-visible appearance can be similar.

Detection. System telemetry on conversation-level cap events and on the success rate of the subsequent continuation turn. The latter is the resume-state quality metric; a high rate of malformed continuations indicates degraded UCT recovery.

Mitigation. Graceful resume design: when a cap fires, the system should preserve a structured progress ledger, summarize work completed, and provide a clean resume entry point when the cap clears. The continuation turn should never degrade to a non-substantive completion such as a literal no-response token.

5.13 Visual Misreading Cascade (VMC)

Definition. Given a structured visual input (chart, table, dashboard, form, schematic, or culturally conventional diagram), the model produces a confident interpretation. When the user corrects a specific structural element, the model produces a new interpretation that is itself incorrect in a different way. The cascade can repeat across multiple correction cycles.

Observed pattern. Sequential confident misreadings of basic structural elements of a structured visual input. Each user correction triggers a fresh interpretation rather than a paused structural confirmation; new errors substitute for old ones until the user explicitly restates the structure. The model never reaches a state where it asks the user to verify chart structure before committing to interpretation.

Why it matters. Multimodal interpretation of structured symbolic inputs (charts, financial statements, diagrams, forms) is a growth area for LLM workflows. VMC implies that the model is not performing a stable first-pass structural extraction before downstream reasoning. The cost compounds because downstream analysis depends on the misread structure.

Differentiation. VMC is distinct from PTS (model does not decline; it produces confident output), from STK (input is user-supplied, not a third-party UI), and from QI (each interpretation is internally consistent; the failure is at the visual-input to symbolic-interpretation boundary).

Detection. Visual-input verification loop: on first interpretation of a structured visual input, the model emits a compact structural summary (field-by-field, line-by-line, cell-by-cell) and explicitly requests confirmation before producing downstream analysis.

Mitigation. Pause-and-confirm protocol on structured visual inputs. The cost is one round-trip; the benefit is elimination of the cascade. Implementable as a model-side reflex or a system-level multimodal pre-check before downstream analysis depends on the reading.

5.14 Artifact Finalization Failure (AFF)

Definition. In artifact-producing workflows, the model generates a deliverable (PDF, DOCX, slide deck, spreadsheet, code file) that nominally exists but does not satisfy user-facing readiness criteria. Terminal text quality is acceptable; artifact-level readiness is not.

Observed pattern. A generated document opens but has misaligned headings, broken links, missing sections, or layout problems that make it unfit for the stated purpose. A generated code file runs but has minor formatting that the user must repair before sharing. The user discovers the failure only on opening or running the artifact.

Why it matters. Terminal text quality is necessary but not sufficient for artifact tasks. Many professional workflows end in deliverables whose acceptance criteria are visual or structural rather than purely textual. A model that produces excellent text but a flawed file imposes recovery cost equal to the gap, and the gap is invisible until the user opens the artifact.

Differentiation. AFF is artifact-readiness-side; PPG concerns specific placeholder substrings inside an otherwise-formatted output; EBC concerns boundary leakage of advisory content. All three can co-occur in a single artifact, and the rubric distinguishes them by what specifically fails the readiness check.

Detection. Automated artifact checks: file opens, page count matches expectation, headings render, no placeholder tokens remain, links resolve, citations support claims, format-specific lints pass. Human review supplements where mechanical checks are insufficient.

Mitigation. A pre-delivery artifact-readiness audit: deterministic checks before emitting the file. Where checks fail, regenerate or surface the failure. Treat the artifact as the unit of evaluation rather than the surrounding text; this is a measurement discipline as much as a model-side change.

5.15 Source Verification Drift (SVD)

Definition. A workflow begins with a requirement to verify sources, claims, citations, or factual assertions, but progressively drifts into plausible synthesis without sufficient source grounding. The drift can be subtle: citations may support adjacent points but not the exact claim, or a source may be stale for a time-sensitive assertion.

Observed pattern. Cited sources support adjacent points but not the exact claim. Citation freshness slips on time-sensitive assertions. The model continues to attach citations to claims that are now partially synthesized rather than directly supported. The presence of citations creates a surface appearance of grounding even when the grounding has weakened.

Why it matters. SVD is corrosive because citations create the appearance of grounding even when grounding has weakened. Downstream consumers of the artifact (reviewers, readers, decision-makers) cannot easily distinguish well-supported claims from drifted ones without re-verifying every citation.

Differentiation. SVD is citation/retrieval-discipline-side. It differs from classical hallucination (which fabricates the claim itself) in that the claim is plausible and the citation is real, but the alignment between them has weakened. It is distinct from QI, which concerns internal arithmetic, and from STK, which concerns stale procedural knowledge.

Detection. Claim-source entailment checks for each cited claim. Link-freshness audits for time-sensitive assertions. Independent verification on a sample of claims, with results compared to the entailment check; large divergence indicates drift.

Mitigation. An explicit verification pass before delivering citation-heavy artifacts: each claim is matched to a source span, and the citation is retained only if the match holds. Where entailment is partial, the claim is softened or removed.

5.16 Editorial Boundary Contamination (EBC)

Definition. Process commentary, advisory metadata, scoring or readiness language, venue strategy, or other chat-level material appears inside a deliverable that should be publication-ready. The artifact-level boundary between conversation and final output fails.

Observed pattern. An intermediate manuscript draft contains a sentence about its own venue suitability. A public summary contains an instruction the user had previously issued in chat. A submitted artifact contains a meta-note about what changed between versions. The user must audit deliverables for advisory content even after substantive content is correct.

Why it matters. Boundary contamination is recoverable but costly because it bleeds private process into public output. The reputational and privacy cost can outweigh the substantive value of the artifact. In professional settings the leakage can be material.

Differentiation. EBC is the artifact-side analogue of the more general distinction between advisory dialogue and final artifact. It is distinct from AFF (which concerns format and readiness) and from SVD (which concerns source grounding). EBC is specifically about boundary management between conversation and deliverable.

Detection. Pre-delivery boundary audit: scan the artifact for explicit advisory language (readiness scores, venue lists, 'you should,' 'I recommend,' process explanations) and for content that should have remained in chat.

Mitigation. An explicit dual-channel protocol: conversation receives advisory content; the artifact receives only content the user asked to be in the artifact. Pre-delivery audit applied automatically to publication-style outputs.

5.17 Audience / Context Miscalibration (ACM)

Definition. The model produces technically correct content that is poorly calibrated for the intended audience, register, or distribution channel. The substantive content is acceptable; the communicative fitness for the audience or channel is not.

Observed pattern. A public-communication artifact contains identifying or personal context that should not appear publicly. A summary written for a general audience uses technical terminology better suited to a specialist audience. A short executive update is delivered as a long, citation-heavy academic paragraph.

Why it matters. ACM is the bridge between substantive content quality and communicative fitness. A model that produces correct content for the wrong audience imposes rework on the user and, in public-communication contexts, can introduce reputational or privacy risk through accidental disclosure.

Differentiation. ACM is audience-side; AFF is format-side; EBC is process-leakage-side. ACM can co-occur with EBC when private process notes leak into an artifact aimed at a public audience, but the two failures are conceptually distinct.

Detection. Audience-and-channel checks: items in the pilot pack require that a summary avoid sensitive personal-context terms and avoid naming specific vendors in audience-restricted artifacts. Boundary terms include identifying personal context, employer-sensitive content, and unrelated personal subject matter.

Mitigation. Explicit audience and channel framing in the prompt or system policy. A pre-delivery audit that checks for terms incompatible with the stated audience, similar in spirit to the EBC audit. The framing should be set by the user, surfaced in the system policy, and enforced by the audit.

6 Cross-Workflow Observations

6.1 Document-generation and artifact-production workflows

When the deliverable is a document, slide deck, spreadsheet, or code file, terminal text quality is necessary but not sufficient. The artifact must open cleanly, render correctly, and meet the user's apply-ready criteria. AFF, PPG, and EBC are concentrated in these workflows; their incidence becomes visible only when the artifact is evaluated outside the chat. A reasonable evaluation

discipline treats the artifact, not the response describing the artifact, as the primary unit of measurement.

6.2 Source-verification and multi-step research workflows

Citation-heavy and verification-heavy workflows surface SVD, PTS, and SMI together. The model may begin with disciplined source-grounding, drift into plausible synthesis as the task lengthens, and selectively escalate retrieval on some claims but not others. Claim-level verification checks are the appropriate measurement; citation presence alone is insufficient.

6.3 Public-communication and audience-targeted workflows

When an artifact is intended for a public or restricted audience, ACM and EBC become salient. Technically correct content can leak private context, off-register tone, or advisory metadata. Mitigation is policy-level rather than model-capability-level.

6.4 Quantitative reasoning workflows

QI surfaces most often when responses contain multiple quantitative claims under shared assumptions. The detection method is mechanical and is currently underused.

7 Pilot Experiment: Retry-Probe Quantification of PTS

7.1 Setup

A fifteen-item heterogeneous retrieval task. Items belong to three structural classes: (1) direct URLs with human-readable paths ($n = 9$); (2) URLs containing opaque numeric identifiers ($n = 3$); (3) title-only references requiring search ($n = 3$).

7.2 Procedure

For each item, first-pass success and retry-after-prompt success were recorded from the verbatim session transcript. The retry prompt ("try a different approach") supplied no new substantive information. Conservative coding: an item counts as an RBF only when the second-pass success demonstrates previously available capability.

7.3 Results

7.4 Exact small-sample statistical analysis

A two-sided Fisher exact test on the 2×2 contingency table (table 4) yields $p = 0.0022$. The one-sided test for the hypothesis that opaque-identifier items have lower first-pass success also yields $p = 0.0022$. The result is statistically meaningful for this item set but should not be read as a population-level rate.

Table 2: Retry-probe pilot results with Wilson 95% confidence intervals.

Metric	Value	95% CI
Total items (N)	15	—
First-pass resolved (p_1)	$12/15 = 80.0\%$	Wilson: $[54.8\%, 93.0\%]$
Resolved after retry (p_r)	$15/15 = 100.0\%$	Wilson: $[79.6\%, 100.0\%]$
Recoverable first-pass failures	$3/15 = 20.0\%$	Wilson: $[7.0\%, 45.2\%]$
Recoverability gap ($G = p_r - p_1$)	+20.0 pp	Discordant CI: $[7.0\%, 45.2\%]$

Table 3: Pilot first-pass performance by item structural class.

Item structural class	n	First-pass resolved	Rate	RBFs
Direct URL (human-readable path)	9	9	100%	0
URL with opaque numeric identifier	3	0	0%	3
Title-only (search-required)	3	3	100%	0

7.5 What the pilot supports and does not support

The pilot supports the *existence* of recoverable behavioral failures in at least one measured retrieval workflow and supports treating first-pass and retry-after-prompt success as separately measurable quantities. It suggests, exploratorily, that opaque numeric identifiers may be a high-incidence trigger for PTS under batch load. It does *not* establish a population-level PTS rate for any model or vendor; it does not establish a universal opaque-identifier-to-failure relationship; and it does not establish a model ranking, because workload and model identity are not experimentally balanced.

8 Intra-Vendor Pilot Results: Sonnet, Opus, and Haiku on the Experiment Pack

To complement the single-batch retry-probe pilot in section 7, we ran the ten-task experiment pack (section 10, Appendix B) against three Anthropic models in parallel: `claude-sonnet-4-6`, `claude-opus-4-7`, and `claude-haiku-4-5-20251001`. The run is an *intra-vendor pilot*: it holds vendor constant and removes one common cross-vendor confound (different prompt-engineering norms, different default tool affordances). Cross-vendor extension to OpenAI and Google model families is deferred to future work because credentials for those vendors were unavailable at the time of this run.

8.1 Setup

Each model received the same ten prompts under standardized settings (max_tokens 800, default decoding; `temperature` omitted for compatibility with the newest Opus model, which deprecates the parameter). Each task was scored by deterministic checks embedded in the task file (forbidden-term lists, required-term lists, or numeric-expectation regexes for two arithmetic items). The pack contains two items per category in five categories: QI (Quantitative Inconsistency), PPG (Phantom Placeholder Persistence), EBC (Editorial Boundary Contamination), ACM (Audience / Context Miscalibration), and RETRY_PROXY (a proxy for stated retry strategy). All 30 calls (10 tasks ×

Table 4: Contingency table for first-pass success by item structural class.

	First-pass success	First-pass failure	Row total
Opaque-identifier	0	3	3
Non-opaque	12	0	12
Column total	12	3	15

3 models) succeeded at the transport level.

8.2 Results

Table 5: Per-(model, category) pass rates with Wilson 95% confidence intervals. Two items per cell.

Model	Category	n	n_{pass}	Pass rate	Wilson low	Wilson high
Sonnet 4.6	ACM	2	2	1.00	0.34	1.00
Sonnet 4.6	EBC	2	2	1.00	0.34	1.00
Sonnet 4.6	PPG	2	2	1.00	0.34	1.00
Sonnet 4.6	QI	2	2	1.00	0.34	1.00
Sonnet 4.6	RETRY_PROXY	2	0	0.00	0.00	0.66
Opus 4.7	ACM	2	2	1.00	0.34	1.00
Opus 4.7	EBC	2	2	1.00	0.34	1.00
Opus 4.7	PPG	2	2	1.00	0.34	1.00
Opus 4.7	QI	2	2	1.00	0.34	1.00
Opus 4.7	RETRY_PROXY	2	0	0.00	0.00	0.66
Haiku 4.5	ACM	2	2	1.00	0.34	1.00
Haiku 4.5	EBC	2	2	1.00	0.34	1.00
Haiku 4.5	PPG	2	2	1.00	0.34	1.00
Haiku 4.5	QI	2	2	1.00	0.34	1.00
Haiku 4.5	RETRY_PROXY	2	0	0.00	0.00	0.66

8.3 Findings

Uniform success on artifact and audience checks. All three Anthropic models passed both items of QI, PPG, EBC, and ACM. This is an existence result for the metrics: the deterministic checks fire cleanly on responses that the rubric considers acceptable, and on these particular tasks all three models produced responses that satisfied the rubric.

Uniform failure on the RETRY_PROXY rubric. All three models also failed both RETRY_PROXY items. Inspection of the outputs and of the deterministic check criteria shows that the failures are largely a *rubric-vocabulary artifact*: the check required specific tokens such as `site`, `exact`, `identifier`, and `alternative` to appear in the response, and the models produced reasonable retrieval-escalation strategies (metadata search, variant formatting, identifier-as-numeric query) that used different vocabulary. The failure is therefore informative about the rubric, not strictly about the models: a model that proposes a sound retrieval strategy in different words is scored as failing.

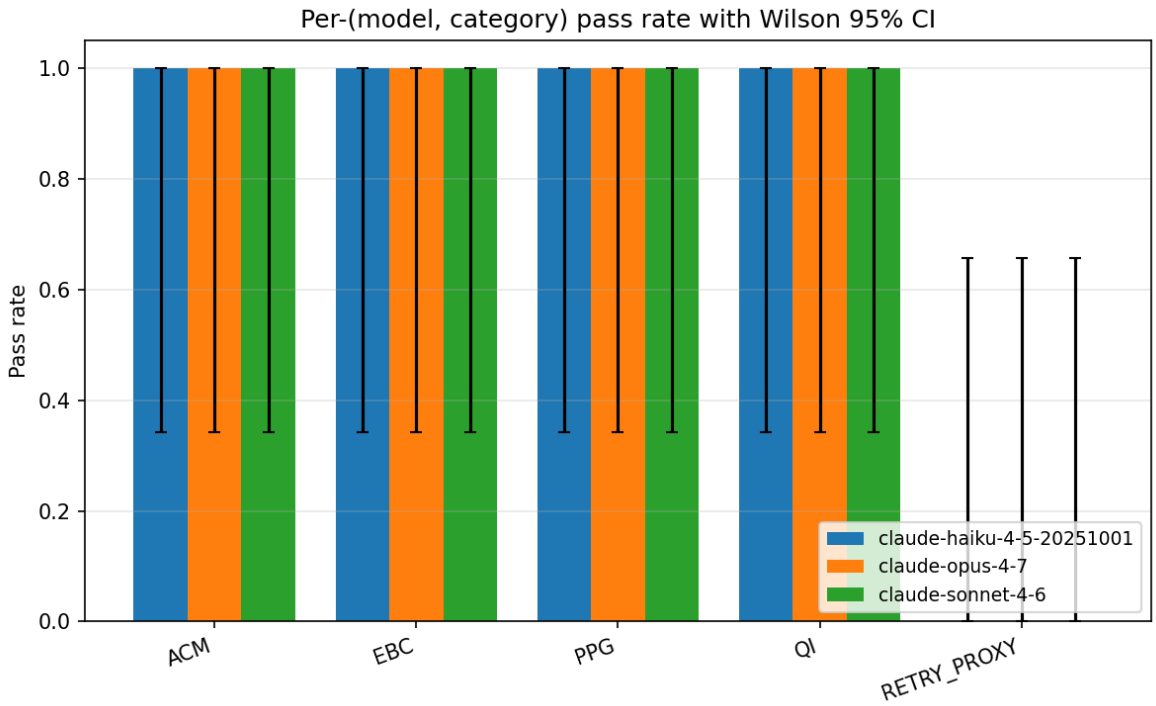

Figure 1: Multi-model pilot pass rate by category with Wilson 95% confidence intervals (intra-vendor). Two items per cell; confidence intervals are wide as expected.

Implication for the RFI mode (section 5). The RETRY_PROXY result is itself a clean instance of why retry-failure invisibility (RFI) is non-trivial to measure. A strict deterministic rubric will systematically miss valid behavior; an LLM-as-judge or a semantic-similarity check would catch the same responses as passes. This is consistent with the broader recommendation in section 10 that automated scoring be paired with double-coded human annotation, especially for retrieval-strategy and persistence-style probes where the space of acceptable answers is open.

8.4 Caveats

The sample is small (two items per cell, ten items per model). Wilson confidence intervals are wide by design. The result is intra-vendor only and cannot speak to cross-vendor differences. The deterministic-check rubric is itself a measurement artifact, as the RETRY_PROXY failure illustrates. We report the pass-rate table to demonstrate that the protocol is implementable end-to-end across multiple frontier models and to illustrate the kind of measurement the §9 protocol would scale up.

9 Observational Notes Beyond the Pilot

Beyond the measured PTS pilot, the case-study corpus contains transcript-verifiable instances of STK, QI, PPG, VMC, UCT, and BDTF. These are reported at the level of pattern, trigger, and recovery so they can be probed in controlled replication. No additional rates are claimed; they are Tier 2 observations supporting taxonomy and protocol design.

10 Proposed Multi-Model Evaluation Protocol

This section specifies a controlled multi-model benchmark grounded in an accompanying ten-task pilot pack across five categories (QI, PPG, EBC, ACM, RETRY_PROXY). Results from running this protocol are not yet available; the protocol is described as a **planned replication**, not a completed study.

10.1 Task families

Retrieval retry probes (PTS, SMI); batch retrieval and scaling (BDTF); quantitative consistency (QI); structured visual interpretation (VMC); third-party UI / tool instructions (STK); phantom placeholders (PPG); artifact finalization and delivery (AFF, PPG, EBC); editorial boundary contamination (EBC); audience / context calibration (ACM).

10.2 Model coverage

Designed for parallel administration across frontier model families: Claude (Sonnet, Opus), GPT, Gemini, and any additional frontier model the investigator can access. Standardized prompts, retry prompts, tool access, and decoding settings are held constant.

10.3 Metrics

First-pass success; retry success; recoverability gap; artifact-readiness rate; numeric consistency rate; placeholder leakage rate; boundary-contamination rate; verification compliance rate; claim-source support rate; human intervention cost.

10.4 Procedure

1. Construct a balanced item set (e.g., 20 direct URLs, 20 opaque-ID URLs, 20 title-only).
2. Administer matched prompts and retry prompts under standardized settings.
3. Hold tool access, temperature, max-tokens, and system prompt constant.
4. Retrieval items at batch sizes $N = 1, 5, 15, 30$.
5. Double-coded annotation with reported Cohen's κ or Krippendorff's α.
6. Report Wilson CIs and exact tests; explicit power statements.

10.5 Sample size and power

For a binary failure rate near 20% with desired margin of error ± 5 pp at 95% confidence, roughly 250 items per condition are required; for ± 3 pp, roughly 700; for ± 2 pp, roughly 1500. Detection of a 10 pp across-model difference at 80% power requires approximately 200 items per arm.

11 Mitigation Implications

- Explicit escalation policy for retrieval (PTS, SMI).
- Batch-aware effort budgeting (BDTF).
- Self-verification gates: arithmetic (QI), claim-source entailment (SVD), file-readiness (AFF, PPG).
- Visual-input verification loop (VMC).
- Staleness-aware tool guidance (STK).
- Pre-delivery boundary audit (EBC); audience-and-channel checks (ACM).
- Graceful resume design (MRT, UCT, MSI).
- Counterfactual-replay evaluation (RFI).

12 Limitations and Threats to Validity

1. Single-user evidence base.
2. Single-coder bias.
3. Inductive taxonomy — may be incomplete; some modes may collapse empirically.
4. Model self-report uncertainty.
5. Missing per-message model attribution in conversation exports.
6. Model-task confound.
7. Single-batch quantitative anchor.
8. Selective observability.
9. Cross-platform abstractions are summary-level.

13 Future Work

1. Run the section 10 multi-model protocol across Claude, GPT, Gemini, and additional model families.
2. Expand to a multi-user corpus with double-coding and reported κ.
3. Longitudinal tracking across model-version releases.
4. Artifact-evaluation infrastructure for downloadability, formatting, placeholder absence, link freshness, boundary contamination.
5. Recommendation to providers: include a model field in conversation-export schemas.
6. Behavioral-failure capture in product feedback flows.

14 Conclusion

Recoverable behavioral failures are cases where a model appears capable of completing a task but fails to exercise that capability on the first pass. They are not captured by standard terminal-output evaluation, because they become visible only when a retry, correction, or artifact audit reveals that the model could have produced the correct output without new substantive information. This paper contributes a definition, a layered sixteen-mode taxonomy, a quantitative retry-probe pilot with exploratory confidence intervals and an exact small-sample association test, abstracted cross-platform observations, and a multi-model evaluation protocol with supporting analysis scripts. AI evaluation should measure not only capability but also persistence, verification, escalation, artifact readiness, and boundary control.

References

Bai, Y., et al. (2022). Training a helpful and harmless assistant with RLHF. *arXiv:2204.05862*.

Chen, W., Wang, X., & Wang, W. Y. (2021). A dataset for answering time-sensitive questions. *arXiv:2108.06314*.

Christiano, P. F., et al. (2017). Deep reinforcement learning from human preferences. *NeurIPS*.

Hendrycks, D., et al. (2021). Measuring massive multitask language understanding. *ICLR*.

Jimenez, C. E., et al. (2024). SWE-bench. *ICLR*.

Liang, P., et al. (2022). Holistic evaluation of language models. *arXiv:2211.09110*.

Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA. *ACL*.

Liska, A., et al. (2022). StreamingQA. *ICML*.

Park, P. S., et al. (2024). AI deception: A survey of examples, risks, and potential solutions. *Patterns*, 5(5).

Sharma, M., et al. (2023). Towards understanding sycophancy. *arXiv:2310.13548*.

Srivastava, A., et al. (2023). Beyond the imitation game (BIG-Bench). *TMLR*.

Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6th ed.). SAGE.

Zhou, S., et al. (2024). WebArena. *ICLR*.

A Observation Bank Coding Schema

Each candidate event in the observation bank is coded with: `observation_id`, `task_family`, `original_prompt`, `first_pass_output`, `failure_criterion`, `retry_prompt`, `new_information_introduced`, `post_retry_output`, `mode_code`, `trigger`, `recovery_type`, `user_visible_cost`, `evidence_tier`. Conservative coding rule: if recovery requires new substantive information, the event is excluded.

B Experiment Pack and Task Schema

The companion pack contains ten items across five categories (QI: 2, PPG: 2, EBC: 2, ACM: 2, RETRY_PROXY: 2), plus runner and scorer scripts for OpenAI, Anthropic, and Google API surfaces. Per-item fields: `id`, `category`, `prompt`, `checks` (forbidden terms, required terms, or category-specific numeric expectations). Scoring uses deterministic checks where possible; remaining items are double-coded by independent annotators.

C Annotation Rubric

1. Did the first-pass output satisfy the task's acceptance criteria? If yes, label *first-pass success*.
2. If failed, did the retry succeed?
3. Did the retry prompt introduce new substantive information? If yes, label *ordinary interactive correction*.
4. If retry succeeded without new information, label *recoverable behavioral failure* and assign a mode code.
5. If retry also failed, label *non-recoverable under available context*.
6. If unclear, label *ambiguous*.

D Reproducibility Checklist

- Item set with stable IDs, prompts, retry prompts, expected outputs, verification methods.
- Model and tool settings recorded.
- Raw outputs archived in append-only JSONL with timestamps.
- Scorer script with deterministic check criteria.
- Annotation guidelines and double-coded labels with reported κ or α.
- Analysis notebook producing all tables, confidence intervals, statistical tests.
- Data statement: collection method, privacy controls, redaction, licenses.
- Versioning: pack version, model versions, analysis-notebook commit hash.